

ESPRIT AI Receptionist

Rapport de suivi de projet – Assistant vocal intelligent

Stage – ESPRIT

Document vivant, mis à jour au fil du projet
Dernière mise à jour : 15 août 2026

Contents

1	Introduction et contexte	3
1.1	Objectif du projet	3
1.2	Contrainte fondamentale : 100% gratuit et local	3
1.3	Répartition de l'équipe	3
2	Architecture globale	4
2.1	Les cinq modules prévus	4
2.2	Récapitulatif des modèles utilisés	4
2.3	Structure du dépôt	5
3	Infrastructure et environnement	6
3.1	Migration du disque C vers le disque D	6
3.2	Cause réelle de l'espace disque C qui se remplissait	6
3.3	Matériel et limites de performance	6
4	Module reconnaissance vocale et détection de langue	7
4.1	Choix techniques	7
4.2	Amélioration : biais de vocabulaire	7
4.3	Protocole de test qualité	7
5	Module RAG (génération de réponses)	8
5.1	Architecture	8
5.2	Amélioration 1 : détection d'ambiguïté entre programmes	8
5.3	Amélioration 2 : qualité des données – fiches "gouvernance"	9
5.4	Amélioration 3 : extraction et regroupement du dossier Admission	9
5.5	Audit de la base de connaissances : fiches peu détaillées ou hors-sujet	9
5.6	Nettoyage : préfixe d'interface web et doublons de contenu	10
5.7	Clôture de l'audit : les 4 chantiers restants traités	10
5.8	Correction : cul-de-sac de la clarification en mode vocal	11
5.9	Nettoyage : catégorie "Vie étudiante"	12
5.10	Consolidation par sujet : facilités de paiement	12
5.11	Consolidation par sujet : réductions sur les frais de scolarité	12
5.12	Nouvelle fiche : attestations et relevé de notes	13
5.13	Écart de vocabulaire : "conseil de classe" vs libellés calendrier	13
5.14	Refonte complète des calendriers : extraction groupée par type d'événement	14
5.15	Confirmation utilisateur : le calendrier semestre 2 est commun à tous les groupes	14
5.16	Règlements de scolarité : ré-extraction par article	15
5.17	Test qualité sur 35 questions et corrections	15
5.18	Contradictions entre sources sur les frais 2024-2025	16
5.19	Test qualité sur 100 questions et confirmation des montants par email officiel	17
5.20	Support arabe/tunisien : traduction avant recherche	17
5.21	Bug NLU : classe fantôme générée par un titre mal parsé	18

5.22	Clarification interactive étendue au chat et au vocal	18
5.23	Réponse toujours en arabe standard, jamais en dialecte tunisien	19
5.24	Fusion des fiches fragmentées : frais de scolarité	19
6	Module synthèse vocale (TTS)	21
6.1	Choix techniques	21
6.2	Réglage de la vitesse	21
6.3	Ajout d'une voix arabe et sélection automatique	21
7	Backend / API REST	22
7.1	Endpoints disponibles	22
7.2	Ce qui reste à faire côté backend	22
8	Application d'ingestion de la base de connaissances (extraction_app)	23
8.1	Contexte et objectif	23
8.2	Pipelines d'extraction par type de source	23
8.3	Extraction des programmes d'étude dans extraction_app	24
8.4	Fusion du brouillon dans la base de connaissances	24
8.5	Constat : "liste toutes les matières" ne remontait qu'une partie	25
8.6	Rebond : confusion entre classes voisines (4e/5e année)	25
8.7	Solution de fond : NLU léger et filtrage par métadonnée	25
8.8	Signalement utilisateur : classes de 1ère à 3ème année sans réponse	27
8.9	Signalement utilisateur : question ECTS sur "4MécaT" sans réponse	28
8.10	Garde-fous obligatoires avant sauvegarde	29
8.11	Interface de validation (/a-valider)	29
8.12	Réindexation à la demande	30
8.13	Page d'historique (/history)	30
8.14	Authentification et planificateur	30
8.15	Intégration avec le dashboard React	31
8.16	Découverte : corruption d'encodage dans les fichiers .docx d'admission	31
8.17	Limites connues de l'extraction des programmes d'étude	32
8.18	Limites assumées	32
9	Dashboard qualité et supervision (/qualite)	33
9.1	Contexte et objectif	33
9.2	Indicateurs calculés	33
9.3	Visualisations variées (compétence dataviz)	33
9.4	Journalisation automatique du trafic réel	34
9.5	Annotation et suppression en temps réel (AJAX)	34
10	Tests et validation	36
10.1	Tests manuels effectués	36
10.2	Test systématique sur 225 questions (scripts/run_quality_test.py)	36
10.3	Scripts de test disponibles	36
11	Limites connues et travail restant	37
11.1	Limites techniques actuelles	37
11.2	Travail restant	37

1. Introduction et contexte

1.1 Objectif du projet

Le projet **ESPRIT AI Receptionist** vise à concevoir un assistant vocal intelligent capable de répondre automatiquement aux appels téléphoniques des étudiants et parents de l'école ESPRIT, en français et en arabe tunisien (dialecte). L'assistant doit pouvoir comprendre une question posée à l'oral, y répondre à partir d'une base de connaissances officielle de l'école, et reformuler cette réponse à l'oral.

1.2 Contrainte fondamentale : 100% gratuit et local

L'ensemble des briques techniques utilisées sont **gratuites et fonctionnent entièrement en local** sur la machine de développement : aucune clé API payante, aucun service cloud facturé à l'usage. Ce choix structure toutes les décisions techniques du projet (choix des modèles, de l'infrastructure, des bibliothèques).

1.3 Répartition de l'équipe

- **Ikbel** : module de génération de réponses (RAG) – préparation de la base de connaissances (FAQ, web scraping à partir d'un mail, règlements, calendrier, admissions, paiements). Procédure de secours si l'information n'est pas disponible : contacter directement les services concernés (Admission : rez-de-chaussée bloc E ; FAQ : réseaux sociaux + réception entrée bloc E ; Calendrier : email sinon service scolarité ; Paiement : service financier, 1^{er} étage bloc E).
- **aziz** : backend / API REST.
- **Auteur de ce rapport** : reconnaissance vocale (STT), détection de langue, synthèse vocale (TTS), et backend / API REST.

À faire, non encore attribué explicitement : dashboard (interface CRUD documents + statistiques/indicateurs) et conteneurisation Docker + déploiement.

2. Architecture globale

2.1 Les cinq modules prévus



Module	Statut
Détection de langue (FR / arabe tunisien)	Fait – faster-whisper, 100% local
Reconnaissance vocale (STT)	Fait – faster-whisper (medium), testé
Compréhension de la question (LLM)	Fait – intégrée au module RAG
Génération de réponses (RAG)	Fait – testé et amélioré (voir chap. 5)
Synthèse vocale (TTS)	Fait – Piper, voix française
API REST (backend)	Fait – 5 endpoints fonctionnels
Dashboard (CRUD + stats/KPI)	Non commencé
Docker (conteneurisation + déploiement)	Non commencé

2.2 Récapitulatif des modèles utilisés

Étape	Modèle / outil	Détails
Détection de langue	faster-whisper (medium)	Même modèle que le STT (partagé en mémoire), CPU, quantification int8
Reconnaissance vocale (STT)	faster-whisper (medium)	100% local, device=cpu, compute_type=int8
Embeddings (recherche RAG)	sentence-transformers (paraphrase-multilingual-mpnet-base-v2)	Multilingue français + arabe, local
Base vectorielle (RAG)	ChromaDB	Stockage des embeddings, persisté dans vector_store/
Génération de réponse (RAG/LLM)	Ollama (qwen2.5:7b-instruct)	7 milliards de paramètres, 50%/50% CPU/GPU (VRAM limitée à 4 Go)

Étape	Modèle / outil	Détails
Synthèse vocale (TTS)	Piper (fr_FR-siwis-medium)	Français uniquement, 100% local

Tous ces modèles sont **gratuits, open-source, et tournent entièrement en local** (aucune clé API), conformément à la contrainte fondamentale du projet (voir section 1.2).

2.3 Structure du dépôt

data/	Collecte et preparation de la base de connaissances
raw_data/	Documents bruts (PDF, pages HTML scrapees)
processed/	Base de connaissances nettooyee (site_esprit_clean.json)
scripts/	Scripts de scraping, extraction PDF, nettoyage
modules/	
rag/	Generation de reponses (RAG)
language_detection/	Detection francais / arabe
speech_to_text/	Reconnaissance vocale
text_to_speech/	Synthese vocale
backend/	API REST reliant les modules
routers/	rag.py, stt.py, tts.py
extraction_app/	Appli separee : ingestion controlee de nouvelles fiches dans site_esprit.json (PDF/Excel/image/URL)
dashboard/	(a venir)
docker/	(a venir)
tests/	Scripts de test manuels
voice_models/	Modeles de voix Piper (gitignore, telechargeable)
vector_store/	Base vectorielle ChromaDB (gitignore, regeneree)

3. Infrastructure et environnement

3.1 Migration du disque C vers le disque D

Le projet a été déplacé du disque C: vers le disque D: (d:\stage_ia_receptionist) par manque d'espace sur C. Les caches de modèles ont été redirigés vers D via des variables d'environnement Windows persistantes :

- `OLLAMA_MODELS = D:\ai_cache\ollama_models\models`
- `HF_HOME = D:\ai_cache\huggingface` (cache Hugging Face : embeddings + Whisper)

Piège identifié : un terminal déjà ouvert avant la création de la variable d'environnement ne la voit pas (les variables User ne sont lues qu'au démarrage d'un nouveau process). Toujours ouvrir un terminal neuf, ou refaire `$env:HF_HOME = "D:\ai_cache\huggingface"` manuellement si un téléchargement inattendu se déclenche.

3.2 Cause réelle de l'espace disque C qui se remplissait

Contrairement à l'hypothèse initiale (les modèles IA), la vraie cause identifiée est double :

1. **Docker Desktop** : les disques virtuels WSL2 (`docker_data.vhdx` $\approx$ 8,7 Go et `ext4.vhdx` $\approx$ 3,5 Go) vivent sur C par défaut et ne rétrécissent jamais automatiquement. Solution proposée (pas encore appliquée) : déplacer l'emplacement du disque Docker vers D.
2. **Fichiers système Windows** : `pagefile.sys` ($\approx$ 19 Go) et `hiberfil.sys` ($\approx$ 6,3 Go), gonflés par la pression mémoire (16 Go de RAM, forte utilisation par Whisper + Ollama + Docker simultanément). Solution proposée : déplacer le fichier d'échange vers D via les paramètres de performance Windows (nécessite droits admin + redémarrage).

3.3 Matériel et limites de performance

- GPU : NVIDIA GeForce GTX 1650 Ti, **4 Go de VRAM seulement**.
- Ollama (modèle 7B) tourne en partage **50% CPU / 50% GPU** (le modèle ne tient pas entier dans la VRAM).
- Whisper (STT) tourne **entièrement en CPU**, décision volontaire pour ne pas entrer en conflit de VRAM avec Ollama (voir `modules/speech_to_text/config.py`).
- Conséquence : `/api/converse` (cycle complet voix→voix) prend environ **20 à 40 secondes** par question, une fois les modèles chargés en mémoire (le tout premier appel après un redémarrage du serveur est plus lent : chargement des modèles en RAM).

4. Module reconnaissance vocale et détection de langue

4.1 Choix techniques

Basé sur `faster-whisper`, modèle `medium`, 100% local et gratuit. Les modules `speech_to_text` et `language_detection` partagent le même modèle chargé en mémoire (`modules/speech_to_text/model.py`) pour éviter de le charger deux fois.

4.2 Amélioration : biais de vocabulaire

Un paramètre `INITIAL_PROMPT` a été ajouté (`modules/speech_to_text/config.py`) pour biaiser le décodage de Whisper vers le vocabulaire du domaine (noms propres ESPRIT, termes admissions/scolarité) et quelques tournures tunisiennes courantes (باش، فم، نجم، باش، Coût nul, réversible, aucune garantie universelle – aide sur des mots spécifiques, pas sur la grammaire ou l'accent en général.

Limite connue et documentée par la recherche : Whisper (même en version large) est entraîné très majoritairement sur de l'arabe standard et quelques dialectes bien dotés en données (égyptien...). Le tunisien (derja) reste sous-représenté. Des modèles communautaires dédiés existent (`sadeemar/whisper-finetuned-Tunisian`, `SalahZa/Tunisian_Automatic_Speech_Recognition`) mais sans garantie de qualité supérieure vérifiée, ou avec un changement d'architecture non négligeable (`WavLM/SpeechBrain`). Un fine-tuning maison demanderait de collecter et annoter nos propres enregistrements tunisiens – hors périmètre réaliste du stage.

4.3 Protocole de test qualité

Un script dédié (`tests/test_language_quality.py`) guide l'utilisateur à travers 8 sujets représentatifs de la base de connaissances (admission, frais, paiement, calendrier, stage, documents, contact général, hors-sujet), enregistre chaque réponse en arabe tunisien, l'enchaîne dans le pipeline complet, et sauvegarde un rapport JSON (`Bureau\test_arabe_resultats.json`) pour analyse structurée.

5. Module RAG (génération de réponses)

5.1 Architecture

- **Base vectorielle** : ChromaDB, persistée dans `vector_store/` (regénérée via `python -m modules.rag.ingest`).
- **Embeddings** : sentence-transformers, modèle paraphrase-multilingual-mpnet-base-v2 (multilingue FR/AR).
- **LLM** : Ollama, modèle qwen2.5:7b-instruct (bon support multilingue parmi les modèles open-source).
- **Seuil de pertinence** : `SCORE_THRESHOLD = 0.45` – en dessous, redirection vers un message de fallback (service humain).

5.2 Amélioration 1 : détection d’ambiguïté entre programmes

Problème constaté : une question vague (“*combien ça coûte pour étudier à ESPRIT ?*”) piochait dans la première fiche la mieux notée, sans vérifier qu’elle correspondait bien au bon programme parmi plusieurs (cours du jour, cours du soir, EMBA...). Exemple observé : réponse de 20 000 DT (tarif EMBA) au lieu des 8 000 DT du programme d’ingénieur généraliste.

Solution implémentée (`modules/rag/programmes.py` + `modules/rag/pipeline.py`) :

1. Chaque fiche est associée à un *programme* (cours du jour tunisiens / internationaux, cours du soir, EMBA, alternance...) déduit de son URL source.
2. Si, parmi les meilleures fiches retrouvées (à une marge de score près), au moins deux programmes différents sont représentés **et** que la question ne précise pas lequel, le système répond directement par une demande de clarification – **sans appeler le LLM** (quasi instantané, ~0,3s au lieu de 15 à 40s).
3. Garde-fou anti-faux-positif : la détection ne se déclenche que si le *meilleur* résultat est déjà spécifique à un programme (sinon, une question sur un service général – stage, carrière... – ne serait pas concernée).
4. Garde-fou supplémentaire : si la question mentionne déjà explicitement un programme par mot-clé (ex. “cours du soir”), pas de clarification demandée.

Résultat validé : questions précises (“frais des cours du soir”) répondent normalement ; questions vraiment vagues (“comment puis-je payer ?”) déclenchent la clarification ; questions sur des services généraux (“service de stage”) ne déclenchent plus de faux positif.

5.3 Amélioration 2 : qualité des données – fiches “gouvernance”

Problème constaté : une question sur le département des stages ne mentionnait jamais le contact réel (Majdi Gharbi) alors que l’information existait dans la base.

Cause racine : `modules/rag/ingest.py` embed le texte `"{titre} : {contenu}"`. Pour les 28 fiches de la page gouvernance, le titre était le **nom de la personne** (ex. “Majdi GHARBI”), qui dilue l’embedding au lieu de l’aider – un nom propre n’apporte aucun signal utile à la recherche sémantique.

Correction appliquée (`data/processed/site_esprit_clean.json`, 28 fiches) :

- **Titre** : nom de la personne → sujet/département (ex. “Département des stages”).
- **Contenu** : reformulé en phrase naturelle (ex. “*Majdi GHARBI est la personne à contacter pour toute question sur département des stages. Email : majdi.gharbi@esprit.tn.*”).
- Base réindexée.

Résultat mesuré : score de similarité passé de absent du top 10 à 0,601 (2^e position) sur la question “qui contacter pour le département des stages ?”. La réponse finale mentionne désormais correctement le contact.

Limite honnête : la correction fonctionne pour les formulations proches du vocabulaire de la fiche. Des formulations plus détournées (“je cherche un stage, qui dois-je contacter ?”) ne remontent toujours pas cette fiche – limite inhérente à la recherche sémantique sur des textes courts.

5.4 Amélioration 3 : extraction et regroupement du dossier Admission

Objectif : les documents `.docx` de `data/raw_data/Admission/` (FAQ par programme, conseillers admission) n’étaient pas encore dans la base de connaissances. Plutôt que d’indexer une fiche par question isolée, le contenu est **regroupé par section** (titres en gras du document, ex. “Admission et Inscription”, “Modalités Financières”) : ni trop court (une seule question, mal retrouvée en recherche sémantique – voir amélioration 2), ni trop long (dilution de l’embedding sur plusieurs sujets différents).

Script : `data/scripts/extract_admission_docx.py` – analyseur souple gérant plusieurs variantes de mise en forme (question/réponse en paragraphes séparés ou combinés, sections purement descriptives), avec nettoyage des notes d’auteur parasites (ex. “(mettre 3 boutons cliquables...)”). Chaque fiche est rattachée au bon programme (réutilisation des URLs canoniques déjà utilisées par le système de détection d’ambiguïté, section 5).

Résultat : 27 fiches ajoutées (785 fiches au total avant l’audit ci-dessous), comparées à `site_esprit_clean.json` existant (aucun doublon exact). Test sur une question vague (“comment ça marche l’inscription aux cours du soir ?”) : réponse complète en 5 étapes – bonne, mais pas systématiquement grâce aux nouvelles fiches précises (beaucoup de contenu existant fait concurrence en recherche).

5.5 Audit de la base de connaissances : fiches peu détaillées ou hors-sujet

Un audit systématique (fiches de moins de 60 caractères, 86 sur 785) a révélé plusieurs catégories de problèmes :

1. **Conflit de calendriers** (~60 fiches) : trois fichiers calendrier différents (1,2,3&4A / 5A / 5DS9&5Infini3) avec des dates différentes pour des événements similaires (ex. vacances d'hiver) – risque de réponse mélangeant les années/promotions. **Pas encore corrigé.**
2. **Bruit PDF hors-sujet** : le document RDI_CATALOG_2023-2024.pdf (rapport annuel de recherche académique – équipes de recherche, publications scientifiques, conférences) s'est révélé **entièrement hors périmètre** pour un assistant de réception admissions/scolarité. **Corrigé** : 49 fiches supprimées (785 → 736 fiches), base réindexée, aucune régression constatée (une des nouvelles fiches "Modalités Financières" est même remontée en 1^{re} position sur un test déjà validé, probablement grâce à moins de contenu parasite en concurrence).
3. **Fiches contact avec nom en titre** (ex. "Ramla Ben Ouirane", "Pr. Faouzi Kamoun") : même défaut que la correction "Département des stages" ci-dessus. **Pas encore corrigé.**
4. **Lignes de grille tarifaire isolées** (ex. "550DT" / "Demi-pension (F1)...") : extraites une par une sans le contexte du tableau d'origine. Lié au sous-dossier Admission/Grilles tarifaires/ (PDF, Excel, zip), **pas encore traité.**

5.6 Nettoyage : préfixe d'interface web et doublons de contenu

Deux problèmes supplémentaires, découverts en creusant le mécanisme même de la recherche sémantique (comment le RAG choisit une fiche pour une question donnée) :

1. **Préfixe d'interface parasite** : 85 fiches sur 736 avaient un titre commençant par "*Ouvrir la visibilité du contenu* : " – le libellé du bouton "déplier" d'un accordéon FAQ, scrapé par erreur avec la vraie question. Comme `modules/rag/ingest.py` donne du poids au titre dans l'embedding, ce préfixe identique sur 85 fiches diluait leur pertinence. **Corrigé** : préfixe retiré (regex simple, aucun risque – ne touche jamais le contenu factuel).

2. **Doublons de contenu préexistants, révélés par le nettoyage** : une fois les titres nettoyés, ces fiches ont mieux remonté en recherche – révélant que **4 groupes de fiches quasi-identiques** existaient déjà entre plusieurs pages de programmes (ex. "Les diplômes délivrés par ESPRIT sont-ils reconnus ?" dupliqué mot pour mot entre les pages "cours-du-jour-tunisiens" et "cours-du-jour-internationaux"). Conséquence concrète : le système de détection d'ambiguïté (section 5) demandait à tort de préciser le programme, alors que la réponse est identique quel que soit le programme choisi. **Corrigé** : les 4 groupes fusionnés en une seule fiche chacun, retaguée avec une source générique non rattachée à un programme spécifique (736 → 730 fiches). Testé et validé : la question "le diplôme ESPRIT est-il reconnu ?" répond désormais directement, sans fausse demande de clarification.

Cet épisode illustre bien la nature itérative du nettoyage de données pour un RAG : corriger un problème (préfixe parasite) peut révéler un autre problème resté invisible jusque-là (doublons), simplement parce que les fiches concernées remontent différemment en recherche une fois nettoyées.

5.7 Clôture de l'audit : les 4 chantiers restants traités

1. **Fiches contact restantes** : 2 dernières fiches avec un nom de personne en titre ("Ramla Ben Ouirane", "Pr. Faouzi Kamoun") corrigées selon le même schéma que "Département des stages". La fiche du centre de carrière a ensuite été reformulée une seconde fois (titre sous forme de question "Qui contacter au centre de carrière ?") après un test montrant qu'elle perdait face à des FAQ génériques "qui contacter" d'autres programmes.

2. Conflit des calendriers : les 67 fiches issues des 3 fichiers calendrier (1,2,3&4A / 5A / 5DS9&Infini3) sont désormais préfixées avec leur promotion (ex. “*Pour 5ème année (5A) – le mardi 2025-12-30 : Vacances d’hiver.*”) : même si plusieurs dates concurrentes remontent en recherche, la réponse ne peut plus mélanger les promotions.

3. Grilles tarifaires (Admission/Grilles tarifaires/) : 7 PDF + 1 Excel extraits avec pdfplumber en mode *tableau* (pas texte brut, pour ne pas mélanger les colonnes). Le fichier zip du dossier s’est révélé être une copie exacte de 4 PDF déjà présents (tailles de fichier identiques) – ignoré. 10 fiches ajoutées : foyer et restauration (tunisiens/internationaux), historique des frais 2018-2025 (cours du jour et cours du soir), classe préparatoire (PREPA, nouveau programme ajouté au système de détection d’ambiguïté), et ESPRIT School of Business (Bachelor/Master). Chaque montant a été vérifié manuellement contre le tableau source avant d’être transcrit en phrase complète – aucune réécriture automatique de chiffres.

4. Champs source et type : vérifié dans le code que ni l’un ni l’autre n’est jamais envoyé au LLM ni à l’embedding (ingest.py et generator.py n’utilisent que titre + contenu). type était réellement mort code (jamais relu) – supprimé de toutes les fiches et des 6 scripts d’extraction. source est en revanche activement utilisé par la détection d’ambiguïté et les citations – conservé. Le vrai risque de confusion identifié était ailleurs : **203 fiches** avaient un titre brut de type “*nom-fichier.pdf - page N*” (ex. Reglement-scolarité-24-25-cs.pdf - page 8), qui pollue l’embedding (titre pesé dans la recherche). Un test empirique a confirmé qu’un titre nettoyé (ex. “*Règlement de la scolarité - Cours du soir 2024-2025*”) score mieux (0,667) qu’un titre brut (0,659) et qu’une absence de titre (0,654) – les 203 titres ont été remappés vers des noms lisibles pour les 18 documents concernés.

5. Robustesse de la détection d’ambiguïté : un cas limite a révélé que la règle “ne se déclenche que si le meilleur résultat a un label” était fragile – un écart de score de 0,001 entre deux fiches (bruit d’embedding) suffisait à faire basculer une question générale (“comment fonctionne le service de stage ?”) en fausse alerte. **Corrigé** : la règle regarde maintenant un consensus sur les 3 meilleurs résultats (majorité avec un label de programme) plutôt que le seul premier résultat – plus robuste au bruit, sans perdre la détection des vrais cas ambigus.

Base de connaissances finale : **740 fiches** (730 avant ce chantier, +10 fiches de grilles tarifaires). Régression complète validée sur 5 scénarios de référence (précis / vague / général / contact / calendrier) avant clôture de ce chantier.

5.8 Correction : cul-de-sac de la clarification en mode vocal

Problème découvert en testant /api/converse avec une vraie voix : sur une question vague (“comment puis-je payer ?”), l’appelant recevait la demande de clarification (“précisez le programme”) *en audio*, sans aucun moyen d’y répondre – /api/converse est un aller-retour unique (audio → audio), sans mémoire de conversation entre deux appels. Demander une précision qu’on ne peut pas recevoir est un cul-de-sac inacceptable en usage téléphonique réel.

Correction : answer_question() accepte désormais un paramètre allow_clarification (modules/rag/pipeline.py). Quand il vaut False, le LLM génère toujours une réponse directe même en cas d’ambiguïté détectée – une note est ajoutée au prompt (modules/rag/generator.py) lui demandant de couvrir les principaux programmes concernés plutôt que de refuser de répondre.

Après test, décision prise d’appliquer ce comportement **uniformément aux 3 endpoints** (/api/ask, /api/voice-ask, /api/converse) : plus aucune demande de clarification nulle part, pour une expérience cohérente quel que soit le canal.

Testé : la même question ambiguë donne désormais une réponse directe et utile sur les 3 endpoints (ex. détail des tranches de paiement pour plusieurs programmes, avec une invitation à vérifier les spécificités de son propre programme), au lieu d’une question sans issue.

5.9 Nettoyage : catégorie “Vie étudiante”

Problème signalé : la catégorie “Vie étudiante” (144 fiches) contenait beaucoup de détail et plusieurs fiches jugées inutiles. Analyse : 103 des 144 fiches étaient en réalité des **règlements de scolarité mal catégorisés** (laissés intacts, car potentiellement utiles pour de vraies questions réglementaires) ; les 41 fiches restantes (clubs, alumni, étudiants internationaux, cellule d’écoute) constituaient la vraie “vie étudiante”, ciblée par ce nettoyage.

Supprimé (9 fiches, aucune valeur informative) :

- 6 fiches “Visite virtuelle” : légendes marketing génériques sans même le nom de la salle décrite (ex. “un espace calme et inspirant”).
- 2 fiches “Règlement intérieur” : renvoyaient simplement à “télécharger le règlement ailleurs”, sans contenu réel.
- 1 fiche fusionnée avec une autre (cellule d’écoute, partie 2/2 courte).

Raccourci (32 fiches : 20 clubs, 3 alumni, 6 étudiants internationaux, 3 cellule d’écoute) : style promotionnel/verbeux retiré, tous les faits conservés (emails, téléphones, adresse, dates de fondation). Médiane de longueur : 821 → 167 caractères.

Erreur de données trouvée et corrigée au passage : la fiche “Cellule d’écoute” contenait une phrase sur la communauté Alumni, copiée par erreur depuis une autre page lors du scraping initial – retirée car factuellement fausse dans ce contexte.

Testé : question sur un club (robotique) → bon club identifié avec email (score 0,722) ; question de détresse psychologique (formulation naturelle) → cellule d’écoute remontée en premier avec coordonnées complètes.

5.10 Consolidation par sujet : facilités de paiement

Demande : quand la question porte sur un sujet transversal (ex. “puis-je payer par facilité ?”), la réponse doit couvrir toutes les méthodes existantes, pas une seule au hasard parmi des doublons.

Constat : le paragraphe “Financement” (prêt Fondation ESPRIT, banques partenaires Amen/Zitouna/UBCI, jusqu’à 36 mensualités) était dupliqué quasi à l’identique sur **6 fiches différentes** (plusieurs pages de grilles tarifaires, années 2024-2025 et 2025-2026). Avec seulement 5 fiches remontées par requête (TOP_K), 4 des 5 places pouvaient être occupées par la même information répétée, au détriment d’informations réellement différentes (ex. le financement spécifique à l’EMBA : 2 à 5 tranches, partenariat Banque Zitouna).

Corrigé : les 6 doublons remplacés par une fiche unique “Facilités de paiement et financement (tous programmes)”, couvrant le cas général (cours du jour/soir) et le cas EMBA côte à côte.

5.11 Consolidation par sujet : réductions sur les frais de scolarité

Constat : l’information sur les réductions (mérite académique, fratrie, cours du soir) était éparpillée sur une douzaine de fiches (FAQ générale, pages Admissions spécifiques par programme, fiches Paiements 2024-2025), avec des taux légèrement différents selon la fiche (25%/10% ici, 30%/15% là) sans qu’il soit évident qu’il s’agissait en réalité de la même politique exprimée sous forme de fourchettes (“excellence haute” 25–30%, “excellence standard” 10–15%) plutôt que de valeurs contradictoires. Aucune fiche ne couvrait le cas des étudiants internationaux

(plafonds 30%/40%), la règle de non-cumul, ni la procédure de demande (dossier conjoint signé par les tuteurs + copies des cartes d'identité).

Corrigé : ajout d'une fiche unique consolidée (`paiements_reductions_scolarite_detail_complet`) couvrant les quatre volets (mérite, fratrie, cours du soir, conditions générales/démarche) à partir d'informations internes confirmées. Les fiches existantes (issues du site, déjà validées) n'ont pas été supprimées – elles restent cohérentes avec la nouvelle fiche (leurs taux sont des cas particuliers dans les fourchettes données), seulement incomplètes prises isolément.

Test de bout en bout après ré-indexation (`python -m modules.rag.ingest`) : la nouvelle fiche remonte en premier (score = 0.876) sur la question “Quelles sont les réductions possibles sur les frais de scolarité ?”. La réponse générée reste volontairement concise (persona vocal, `SYSTEM_PROMPT` impose des “réponses courtes et naturelles à l'oral”) et ne récite donc pas exhaustivement tous les pourcentages – comportement voulu pour un appel téléphonique, mais qui mérite d'être validé avec l'utilisateur si le besoin réel est une réponse plus détaillée pour ce type de question transversale.

5.12 Nouvelle fiche : attestations et relevé de notes

Ajout d'une fiche FAQ (`faq_attestation_presence_reussite_releve_notes`) sur la procédure pour obtenir une attestation de réussite, une attestation de présence ou un relevé de notes – information communiquée directement par l'utilisateur (contact par email au service des élèves correspondant à la classe, une adresse par année de 1^{re} à 5^e, puis passage au bureau du service des élèves trois jours après l'envoi). Vérifié : aucune fiche existante ne couvrait déjà ce sujet, pas de risque de doublon.

5.13 Écart de vocabulaire : “conseil de classe” vs libellés calendrier

Constat : la question “Quelle est la date des conseils de classe ?” ne remontait **aucune** des fiches calendrier pertinentes (meilleur score 0,69, sur une fiche totalement hors-sujet), alors que la question “Quelle est la date du conseil de la session principale ?” fonctionnait (score 0,765). Cause : les 4 fiches calendrier concernées utilisent exclusivement les libellés bruts du PDF source (“Conseils Session Principale”, “Conseils de la session principale”) et ne contiennent jamais le mot “classe” – le terme générique que les appelants utilisent naturellement n'existe nulle part dans le texte embeddé de ces fiches.

En vérifiant les PDF sources (`pdfplumber`, avant toute correction) : le libellé “Conseils de la session principale” apparaît bien deux fois à l'identique dans `Calendrier_5DS9 & 5INFINI3.pdf` (16 et 30 janvier) – confirmé fidèle au PDF, pas une erreur d'extraction, donc non modifié. En revanche, la coquille “Session de rattraoage” (pour “rattrapage”) est également confirmée présente telle quelle dans le PDF source (`Calendrier_5A_2526_S1.pdf`) – corrigée dans la base car elle nuit à la recherche sans changer le sens.

Corrigé : les 4 fiches calendrier mentionnant un conseil (2 pour 5^{ème} année/5A, 2 pour 5DS9/5Infini3) enrichies pour inclure explicitement “conseil de classe” en titre et contenu, en plus du libellé original conservé.

Test après ré-indexation : la question “conseils de classe” fait maintenant remonter les deux fiches 5A (scores 0,679 et 0,635) et la réponse générée donne correctement les deux dates (3 décembre : session principale, 24 décembre : rattrapage). Point de vigilance : les scores restent inférieurs à certains résultats hors-sujet (0,69/0,68) – la correction fonctionne mais reste proche du seuil, une base plus large de synonymes (ex. mentionner aussi “conseil de classe” dans les fiches 1^{re} à 4^e année si un tel évènement existe dans leur calendrier) resterait à vérifier si le problème réapparaît sur d’autres formulations.

5.14 Refonte complète des calendriers : extraction groupée par type d’événement

Demande : l’ancienne extraction (`data/scripts/extraction_calendrier.py`) produisait une fiche minuscule par événement individuel (une par jour), jugée insatisfaisante. Un nouveau fichier a par ailleurs été ajouté (`Calendrier du deuxième semestre_2526.pdf`), à intégrer également.

Nouveau script : `data/scripts/reextract_calendrier_groupe.py`. Reprend la reconstruction de date fiable (extraction de tableaux `pdfplumber`, déjà éprouvée), mais regroupe les événements par **type** (rentrée/débuts de semestre, APP0, examens session principale, examens session de rattrapage, conseils de classe, proclamation des résultats, fin des cours, vacances, soutenances, autres) plutôt qu’un par jour – une seule fiche cohérente par (groupe d’année × type) listant toutes les dates correspondantes.

Bug d’extraction découvert et corrigé : en inspectant la structure brute des tableaux `pdfplumber` (pas seulement le texte à plat), certains événements dont le libellé est trop long pour une seule case (ex. “Examens de la Session de Rattrapage”) débordent visuellement sur la case du jour **suivant** dans le PDF source – `pdfplumber` attribue alors à tort le second morceau de texte (“Rattrapage” seul) à ce jour suivant, créant deux événements fantômes tronqués au lieu d’un seul événement complet correctement daté. Corrigé par une fusion des fragments de continuation reconnus (`CONTINUATION_FRAGMENTS`) avec l’événement de la veille, quand les deux jours sont consécutifs. Egalement filtrés : les numéros de semaine (“S1” à “S16”) affichés dans la grille, qui ne sont pas de vrais événements.

Les 45 anciennes fiches “Calendrier*” supprimées de `site_esprit_clean.json` et `site_esprit.json`, remplacées par 34 fiches regroupées (ajoutées en fin de fichier). Après ré-indexation, trois questions testées avec de bons scores : date de l’APP0 (0,783), début des études 5^{ème} année (0,751), conseils de classe (0,771 – contre 0,679 avec la version précédente, et 0,69 sur une fiche hors-sujet avant tout correctif). La réponse sur les conseils de classe mentionne aussi correctement l’absence de date pour les années 1^{re} à 4^e (vérifié : ce calendrier ne contient effectivement aucun événement “conseil”).

5.15 Confirmation utilisateur : le calendrier semestre 2 est commun à tous les groupes

Point laissé ouvert plus haut : confirmé par l’utilisateur que `Calendrier du deuxième semestre_2526.pdf` s’applique de la même façon aux 5 groupes d’année (1^{re} à 5^e), malgré l’absence de qualificatif dans son titre. Les événements de ce fichier sont donc désormais fusionnés dans les fiches de **chacun** des 3 groupes existants (au lieu d’une catégorie à part), avec la source combinée des deux fichiers d’origine par fiche concernée.

Bug de génération découvert en aval : même après cette fusion, la question “Quand commence la session de contrôle pour les 1ère année ?” produisait toujours une réponse vague et non sourcée (“généralement en février ou mars... vérifiez le site web”), alors que la fiche pertinente était bel et bien récupérée (2^e sur 5 sources, score 0,692) – le LLM ne faisait simplement pas le lien entre “session de contrôle” (terme de l'utilisateur) et “session de rattrapage” (terme utilisé dans la fiche), et ignorait cette source dans sa réponse malgré sa présence dans le contexte fourni. Corrigé par le même type de synonyme explicite que pour “conseil de classe” : les fiches de type rattrapage portent désormais aussi la mention “(session de contrôle)”.

Après ce correctif et une nouvelle ré-indexation : la fiche concernée remonte en **premier** résultat (score 0,706, contre 2^e position à 0,692 avant), et la réponse générée cite désormais une vraie date de la base (“la session de contrôle commencera le mercredi 24 juin 2026”) au lieu d'une réponse générique inventée.

5.16 Règlements de scolarité : ré-extraction par article

Constat le plus important de cette session : les 103 fiches de règlement (cours du jour, cours du soir, alternance, Private School en anglais) étaient en réalité des **pages brutes de PDF** (une fiche = une page, 100 à 2000 caractères), et non des fiches organisées par sujet – une page pouvant couper un article en plein milieu ou mélanger la fin d'un article et le début du suivant. Cause : ces 4 documents avaient été traités par le scraper web générique (extraction page par page), qui ignore la structure “Article N : Titre” pourtant bien présente dans les PDF sources.

Correction : nouveau script `data/scripts/reextract_reglements.py`, qui réextrait les 4 documents en respectant leur structure par article :

- Découpage sur le motif `Article N :`, suppression du sommaire (table des matières) et des lignes de sommaire à pointillés (ASCII . ou ellipses unicode ...) qui polluaient certains articles.
- Les articles trop longs (ex. “Régime des examens”, “Charte de l'étudiant”, 8 000 à 13 700 caractères – ils regroupent des sous-clauses non numérotées) sont sous-découpés en parties de 2000 caractères maximum, comme pour les autres documents du projet.
- Dédoublonnage des titres d'article répétés par erreur dans le PDF source.

Doublon supplémentaire découvert et supprimé : 23 fiches “cours du jour” existaient déjà, issues d'une extraction antérieure non documentée (source = nom de fichier local, sans URL) – contenu identique à la nouvelle extraction, supprimées au profit de la version cohérente avec le reste de la base (source en URL `https://`).

Résultat : 103 fiches fragmentées → **88 fiches cohérentes par article et par parcours**. Base de connaissances finale : **685 fiches**. Testé : question sur les conditions d'obtention du diplôme (cours du jour) → score de pertinence 0,782 (contre absence de structure claire avant), réponse complète et exacte citant l'article source.

5.17 Test qualité sur 35 questions et corrections

Un test systématique sur 35 questions couvrant 6 catégories (paiements, calendrier, règlements, admissions, vie étudiante, questions pièges) a été mené via `answer_question()` directement (script `_test_batch_questions.py`, temporaire). Deux problèmes réels ont été détectés et corrigés :

1. Mauvaise identité du directeur. À la question “quel est le nom du directeur actuel d'ESPRIT ?”, le modèle répondait *Lamjed Bettaieb (Directeur Général Adjoint)* au lieu du vrai Président-Directeur Général, *Tahar Ben Lakhdar*. Cause : 9 fiches de gouvernance différentes (“Directeur administratif et financier”, “Directeur des études”, “Directeur Général Adjoint”...) contiennent toutes le mot générique “Directeur” et scorent de façon quasi identique (0,475 à 0,561) – le LLM, recevant les 5 meilleures sans distinction claire de hiérarchie, ne priorisait pas le Président-Directeur Général. **Corrigé** : ajout d'une fiche dédiée “Qui est le directeur de l'école ESPRIT ?” répondant sans ambiguïté – score de similarité 0,656 (nettement au-dessus des 9 fiches concurrentes), résout le cas.

2. Réponse totalement corrompue (bug sévère). À la question “combien de fois je peux redoubler ?”, le modèle générait un texte **en chinois**, sans rapport avec la question (une conversation inventée sur la météo). Cause : le score de la meilleure fiche récupérée n'était que 0,451 (à peine au-dessus du seuil de fallback 0,45), noyé parmi des fragments de prix totalement hors-sujet (“20 000 DT TTC”, “3 200DT”...) – le vrai fait (“le redoublement n'est toléré qu'une seule fois par cycle de formation”) existait bien dans la base mais enfoui dans un article de règlement de 1856 caractères, mal capté par la recherche sémantique. **Corrigé** : ajout d'une fiche dédiée consolidant les 3 parcours, avec une phrase de désambiguïsation explicite (“un seul redoublement au total est autorisé, jamais deux”) après qu'une première version ait fait dire au LLM “deux redoublements” par mauvaise lecture d'une formulation par-parcours. Score final : 0,536 (validé empiriquement avant mise en production, comme pour les autres corrections de ce rapport).

Leçon retenue : un score de similarité proche du seuil de fallback (0,45) est un signal fiable de risque de génération de mauvaise qualité, y compris des dérives complètes (langue incorrecte, hors-sujet) – pas seulement des réponses imprécises. À surveiller pour d'éventuels ajustements futurs du seuil ou de garde-fous sur la génération.

5.18 Contradictions entre sources sur les frais 2024-2025

Un script de nettoyage externe (non écrit dans cette session) proposait de supprimer les fiches issues d'un email de paiement, jugées “fausses” par rapport à la grille tarifaire du site. Vérification avant exécution – deux cas distincts :

Cas 1 (résolu par suppression) : 4 fiches (*admissions_cours_du_jour_tunisiens_003* à *006*, valeurs de 2ème tranche par année) ne correspondaient pas exactement à la fiche historique sourcée depuis le PDF officiel de grille tarifaire (3 des 4 valeurs différaient de 50 à 100 DT). Contrairement à ce qu'affirmait le script, ce n'étaient pas des doublons identiques – mais le PDF officiel étant plus fiable qu'une page web potentiellement obsolète, les 4 fiches ont été supprimées.

Cas 2 (résolu par clarification, pas suppression) : le taux de réduction pour mérite académique différait entre l'email (30%/15%) et le site (25%/10%). Lecture attentive du contenu du site : celui-ci précise explicitement que le taux 25%/10% est *nouveau et applicable aux frais de 2025-2026*, décidé pendant l'année 2024-2025. Le taux de l'email est donc probablement correct pour 2024-2025, pas une erreur. Les deux valeurs ont été **conservées**, avec les titres clarifiés par année plutôt que l'une supprimée au profit de l'autre.

Cas 3 (non tranché, signalé pour vérification humaine) : le montant total des frais 2024-2025 diffère aussi (8200 DT selon l'email vs 8400 TND selon le site et une FAQ docx). Contrairement au cas 2, **aucune des fiches ne mentionne de réserve d'année** – les deux affirment ce montant pour la même année 2024/2025 sans nuance. Faute de moyen de vérifier laquelle est correcte, **aucune fiche n'a été supprimée ni fusionnée** : les deux valeurs restent dans *site_esprit_clean.json*, et le conflit est documenté séparément dans *data/processed/conflits_a_verifier* pour vérification manuelle auprès du service financier.

Principe appliqué : ne jamais trancher automatiquement une contradiction de données factuelles (montants, taux) sans preuve claire de laquelle est correcte – soit clarifier par un facteur objectif retrouvé dans le contenu (ici, l'année d'application), soit signaler explicitement pour vérification humaine plutôt que de deviner.

5.19 Test qualité sur 100 questions et confirmation des montants par email officiel

Un second test systématique, sur 100 questions couvrant 10 catégories, a révélé que le bug de dérive complète du LLM (section 5.17) n'était pas propre à la question du redoublement : il est réapparu sur une question sans rapport (classe préparatoire), confirmant qu'il s'agit d'un risque systémique lié à un score de pertinence faible, pas d'un cas isolé. Deux autres réponses se sont révélées fausses ou contradictoires (campus de Monastir, existence d'ESPRIT School of Business), à recouper avec les catégories internes de programmes.

L'utilisateur a ensuite fourni les deux emails officiels de réinscription (2024-2025 et 2025-2026) envoyés par la Direction, résolvant définitivement le conflit du montant total signalé plus haut : **8200 DT pour 2024-2025 est confirmé correct** (les fiches affirmant 8400 TND, aujourd'hui corrigées, étaient fausses) ; **8350 TND pour 2025-2026** est ajouté comme nouvelle donnée confirmée (4050 TND avant le 31/08/2025, 4300 TND avant le 15/01/2026), avec 7 moyens de paiement détaillés (en ligne, Amen Bank, BIAT, chèque, sur place, prélèvement 10 mensualités, traite avalisée).

Difficulté technique rencontrée et résolue : les nouvelles fiches 2025-2026, formulées de façon neutre, scoraient moins bien (0,68) que l'ancienne fiche 2024-2025 (0,85) pour une question portant explicitement sur 2025/2026 – confirmation que les embeddings distinguent mal les années pures, un chiffre à 4 chiffres se ressemblant sémantiquement d'une année à l'autre. Corrigé en répétant l'année plusieurs fois dans le contenu et en contrastant explicitement avec l'année précédente ("concerne spécifiquement 2025/2026, différent de 2024/2025") : score porté à 0,88, redevenant la source la plus pertinente. Le `conflits_a_verifier.json` a été mis à jour pour marquer ce conflit comme résolu.

5.20 Support arabe/tunisien : traduction avant recherche

Problème constaté : une question posée en arabe pur (ou en dialecte tunisien) matchait nettement moins bien ($\sim 0,08$ de similarité cosinus en moins, mesuré sur un cas réel) la fiche pertinente que son équivalent français – la base de connaissances étant presque entièrement en français, l'écart suffisait à faire sortir la bonne fiche du `top_k` et faire répondre le modèle à côté du sujet.

Solution implémentée (`modules/rag/translate.py`) : la question est traduite en français par le LLM local **avant** la recherche par similarité (pas après), avec une température fixée à 0 (traduction déterministe, pas créative – un essai avec la température par défaut avait produit une traduction mêlant du texte traduit correct à une remarque hallucinée). Un lexique de référence est injecté dans le prompt de traduction pour forcer le vocabulaire administratif exact d'ESPRIT plutôt qu'une traduction littérale (ex. "*panier*" pour `بانيني`, "*classe*" pour `قسم` suivi d'un code, "*payer*" – pas "*finir*" – pour `نخلص` en tunisien). La réponse finale, elle, est générée à partir de la question **originale** (non traduite), pour répondre dans la langue effectivement utilisée par l'appelant.

Cas résolus grâce à ce lexique : plusieurs mots tunisiens très fréquents (قداش = combien, نخلص = payer, اسبندراك دروس = cours de soutien) étaient mal traduits par défaut, entraînant une recherche sur un sujet totalement différent de la question posée (ex. “comment payer mes frais depuis l'étranger” traduit en “comment terminer mes études et revenir en Tunisie”). Chaque cas a été diagnostiqué en comparant la traduction produite au sens réel de la question tunisienne, puis corrigé en ajoutant l'entrée manquante au lexique.

5.21 Bug NLU : classe fantôme générée par un titre mal parsé

Problème constaté : une question generale contenant “à ESPRIT” (ex. “Combien d'absences sont tolérées avant sanction à ESPRIT ?”) filtrait à tort la recherche sur une classe précise du programme d'études, masquant la bonne réponse.

Cause racine : une fiche de programme d'étude mal titrée (“3B - 2025/2026 Semestre - 5”) génère un nom de “classe” contenant le mot “Semestre”, dont un alias auto-généré (concaténation des 2 premiers caractères de chaque mot significatif, ex. “bse”) matchait par **sous-chaîne** n'importe quel mot le contenant – “bse” est littéralement une sous-chaîne de “absences”. Un bug de la même famille (alias d'1 seul caractère, ex. “a” pour la classe “3A”, matchant le mot “à”/“a” omniprésent en français) avait déjà été corrigé plus tôt dans le projet ; celui-ci en est une variante non couverte par le correctif initial.

Correction (modules/rag/nlu.py) : les classes dont le nom contient le mot “semestre” (artefact d'extraction, jamais un vrai nom de classe qu'un appelant mentionnerait) sont désormais exclues de la liste des classes reconnues. La regex de nettoyage d'année dans `_classe_core()` a aussi été corrigée (elle cherchait un tiret/slash séparateur qui avait déjà été remplacé par un espace en amont, donc ne s'appliquait jamais).

5.22 Clarification interactive étendue au chat et au vocal

Le mécanisme de détection d'ambiguïté entre programmes (section 5.17 et § “Amélioration 1”) existait déjà dans `modules/rag/pipeline.py`, mais n'était **activé sur aucun canal réel** : les trois points d'entrée (`/api/ask`, `/api/voice-ask`, `/api/converse`) appelaient `answer_question(..., allow_clarification=False)`, désactivant systématiquement la demande de précision au profit d'un mélange des programmes concurrents dans une même réponse (chacun nommé explicitement, mais sans garantie que le LLM suive bien cette consigne).

Activation (sur demande explicite, avec validation préalable des canaux concernés) :

- `/api/ask` (chat texte) : `allow_clarification=True`.
- `/api/voice-ask` (démon vocal) : `allow_clarification=True`, avec un nouveau paramètre `pending_question` (formulaire) permettant au frontend de renvoyer la question d'origine mémorisée, recombinaison côté serveur avec la réponse orale de l'appelant avant un nouvel appel au pipeline.
- `/api/converse` (canal audio-vers-audio, pas encore branché à un véritable système téléphonique) : laissé inchangé, faute de mécanisme de mémoire de conversation exploitable pour ce canal.

Mémoire côté client (`dashboard/src/components/ChatDemo.tsx` et `VoiceDemo.tsx`) : le chat et le vocal étant sans état côté serveur, chaque composant retient la question d'origine dans une référence locale quand `needs_clarification=true`, et la recombine avec la réponse de l'utilisateur (“{question} ({précision})”) avant le prochain appel – sans quoi le serveur

recevrait uniquement “Tunis” ou “cours du jour”, sans savoir à quelle question d’origine cela se rapporte.

Deux bugs découverts en testant le flux complet :

1. **Fiches de règlement invisibles à la détection d’ambiguïté.** Les fiches scrapées depuis des PDF de règlement (ex. “Cours du soir - Article 4”) ont une URL source qui ne contient aucun segment reconnu par `programme_label()` (basé sur l’URL des pages web), même quand leur **titre** identifie clairement le programme concerné. Conséquence réelle : une question générale sur les horaires (“à quelle heure commencent les études ?”) ne déclenchait jamais de clarification jour/soir, car seule la fiche EMBA (URL reconnue) était détectée comme “programme”, l’autre (cours du soir, PDF) restant invisible. **Corrigé** : `programme_label()` accepte maintenant un titre en repli, avec une table de préfixes de règlement (“Cours du jour - Article ”, “Cours du soir - Article ”...) vérifiée si la recherche par URL échoue.
2. **Risque de boucle infinie de clarification.** Une fois le bug précédent corrigé, un nouveau cas est apparu : si l’appelant répond avec un programme que la recherche n’a **pas** détecté dans son ensemble ambigu initial (ex. “cours du jour” alors que seuls “cours du soir” et “EMBA” avaient été proposés), le système redemandait indéfiniment la même précision. **Corrigé** (`modules/rag/pipeline.py`) : la vérification “la question mentionne-t-elle un programme ?” se fait désormais aussi contre l’ensemble **complet** des programmes connus (pas seulement ceux détectés dans ce tour précis) – dès que l’appelant nomme explicitement un programme reconnu, le système lui fait confiance plutôt que de re-questionner le classement de la recherche.

Limite assumée, non corrigée : même après ces deux correctifs, certaines questions vraiment ambiguës (ex. “à quelle heure commencent les études ?”) ne déclenchent toujours pas de clarification, car la recherche par similarité ne fait jamais remonter suffisamment de fiches distinctes dans le `top_3` utilisé pour le consensus (`TOP_N_FOR_CONSENSUS`). Décision explicite de garder ce seuil de sensibilité tel quel pour l’instant plutôt que de le rendre plus agressif (risque de clarifications inutiles) – à ajuster si ce type de cas se répète en usage réel.

5.23 Réponse toujours en arabe standard, jamais en dialecte tunisien

Sur demande explicite (minimiser les erreurs de langue en sortie), la consigne de langue du prompt système (`modules/rag/generator.py::SYSTEM_PROMPT`) a été resserrée : une question en français reçoit une réponse en français (inchangé) ; une question en arabe – standard **ou** dialecte tunisien, mélangé au français ou non – reçoit désormais systématiquement une réponse en **arabe standard** (فصحى), jamais en dialecte, avec une consigne explicite de ne pas imiter le registre de la question posée. Validé sur une question réelle en tunisien (□□□□ □□□□ □□ ”□□□□□□) : la réponse produite ne contient plus aucun marqueur de dialecte tunisien.

5.24 Fusion des fiches fragmentées : frais de scolarité

Problème constaté : à la question “quels sont les frais d’inscription à ESPRIT (cours du jour) ?”, le modèle ne mentionnait que la 1^{ère} tranche des frais, omettant le montant total et la 2^{ème} tranche.

Cause racine : le montant total et chaque tranche existaient dans **3 fiches séparées** (`paiements_2024-2025_montant_total`, `_tranche1`, `_tranche2`), en concurrence avec une fiche

déjà complète existant par ailleurs pour 2024-2025 – selon la formulation exacte de la question, la recherche pouvait ne remonter que la fiche “tranche 1” seule dans le `top_k`.

Correction : les 3 fiches fragmentées 2024-2025 (redondantes avec la fiche complète) ont été supprimées ; les 3 fiches fragmentées 2025-2026 (pas de fiche complète équivalente) ont été fusionnées en une seule, avec le même format (montant total + 2 tranches datées). Le mot “frais d’inscription”, absent du vocabulaire des fiches existantes (qui utilisaient uniquement “frais de scolarité”), a été ajouté comme synonyme explicite dans le titre et le contenu.

Limite non résolue : la formulation informelle exacte issue du flux de clarification (§ 5.22), “frais d’inscriptions... (*cours du jour*)”, continue à mal retrouver l’information même après ces corrections – le modèle d’embeddings confond cette formulation précise avec des fiches de frais de foyer (résidence étudiante) ou d’EMBA. Plusieurs pistes de contenu ont été essayées (dont une modification de titre par négation, qui a **empiré** le score – les modèles d’embeddings ne traitent pas la négation comme un contraste sémantique, ils la traitent comme des mots supplémentaires qui rapprochent). Un vrai correctif demanderait un filtrage de la recherche par métadonnée “programme” (comme il en existe déjà pour “classe” et “campus”), pas encore implémenté pour cet axe.

6. Module synthèse vocale (TTS)

6.1 Choix techniques

Basé sur **Piper** (moteur de synthèse neuronale, 100% local, gratuit, aucune clé API). Voix française `fr_FR-siwis-medium` installée dans `voice_models/` (gitignored, retéléchargeable).

6.2 Réglage de la vitesse

La voix par défaut parlait trop vite pour un usage téléphonique. Un paramètre `SPEECH_LENGTH_SCALE = 1.25` (25% plus lent) a été ajouté dans `modules/text_to_speech/config.py`, appliqué via `SynthesisConfig` de Piper.

6.3 Ajout d'une voix arabe et sélection automatique

Une voix arabe standard (`ar_JO-kareem-medium`, arabe jordanien – aucune voix tunisienne disponible chez Piper) a été ajoutée en complément de la voix française. Le module sélectionne automatiquement la voix à utiliser selon la langue détectée du texte à synthétiser (voir `modules/text_to_speech/synthesize.py`), cohérent avec la règle du pipeline RAG qui répond désormais toujours en arabe standard, jamais en dialecte tunisien (aucune voix dialectale n'aurait de toute façon été disponible pour la lire).

Limite restante : la voix arabe disponible est jordanienne, pas tunisienne – l'accent ne correspond pas à celui des appelants. Aucune voix Piper tunisienne n'existe à ce jour ; limite structurelle de l'écosystème open-source disponible, pas du projet lui-même.

7. Backend / API REST

7.1 Endpoints disponibles

Endpoint	Description
GET /api/health	Vérifie que l'API répond.
POST /api/ask	Pose une question texte au RAG. Renvoie réponse + sources + indicateurs (used_fallback, needs_clarification).
POST /api/transcribe	Envoie un fichier audio, renvoie langue détectée + transcription.
POST /api/voice-ask	Envoie un fichier audio contenant une question orale, renvoie la réponse générée par le RAG (texte).
POST /api/speak	Envoie un texte, renvoie l'audio (.wav) synthétisé.
POST /api/converse	Cycle complet : audio (question) → transcription → RAG → audio (réponse).

7.2 Ce qui reste à faire côté backend

- Gestion d'erreurs explicite (fichier audio corrompu, texte vide, format non supporté).
- Tests automatisés (actuellement : validation manuelle via `curl` et `/docs`).
- Endpoints pour le futur dashboard (CRUD documents, statistiques).

8. Application d'ingestion de la base de connaissances (extraction_app)

8.1 Contexte et objectif

Le pipeline de collecte existant (data/scripts/) est un ensemble de scripts lancés manuellement, un par source, sans filtrage de pertinence, sans détection de doublon et sans journal d'activité. Pour permettre d'ajouter de nouvelles fiches à la base de connaissances de façon plus contrôlée – et sans risquer de polluer `site_esprit_clean.json` avec du contenu hors-sujet ou déjà présent – une application web séparée a été développée : `extraction_app/`. Elle vit dans un dossier indépendant du reste du projet (ne touche ni `backend/`, ni `modules/`, ni `data/scripts/`) et tourne sur un port distinct (8001) pendant que le backend principal reste sur le port 8000.

8.2 Pipelines d'extraction par type de source

Source	Logique
PDF	Trois niveaux, dans l'ordre : (1) programme d'étude – détection d'un tableau UE/panier + matières + heures/période/ECTS (réutilise <code>data/scripts/extract_programmes_etude.py</code> , une fiche par classe×panier) ; (2) Article N : si la structure existe (réutilise <code>data/scripts/reextract_reglements.py</code> – retrait du sommaire, découpage des articles trop longs) ; (3) sinon repli sur une fiche par page.
Word (.docx)	<code>python-docx</code> , regroupement par section (dernier titre en gras rencontré), détection des paires question/réponse à l'intérieur d'une section (réutilise la logique de <code>data/scripts/extract_admission_docx.py</code>) – une fiche par sujet complet plutôt que par mini-question isolée.
Excel (.xlsx)	<code>pandas/openpyxl</code> , une fiche par ligne pertinente de chaque feuille (concaténation colonne: valeur).
Image (.jpg/.png)	OCR <code>pytesseract</code> , puis même découpage que le PDF. Si le texte reconnu est trop court ou contient trop de caractères non reconnus, la fiche n'est jamais fusionnée silencieusement – elle est mise de côté dans <code>data/a_verifier.json</code> pour vérification manuelle.

Source	Logique
URL	requests + BeautifulSoup, structure FAQ-accordéon dédiée pour esprit.tn (réutilise la logique de data/scripts/web_scraping_espace_etudiant.py, y compris le nettoyage des libellés d'accordéon), extraction générique par sections h2/h3 sinon.

8.3 Extraction des programmes d'étude dans `extraction_app`

Suite à l'extraction en masse des 30 PDF de programmes d'étude (registre de validation publié pour revue – voir `data/processed/programmes_etude_a_valider.json`), la même logique a été intégrée directement dans le pipeline PDF de `extraction_app` (`services/programme_etude_extractor.py`) pour que tout nouveau programme d'étude uploadé soit traité automatiquement de la même façon, sans script séparé à relancer manuellement.

Testé de bout en bout via l'interface (upload de `SAE.pdf`, non testé auparavant) : 17 fiches ajoutées (une par classe×panier), 0 rejet hors-sujet (le texte contient naturellement "programme d'étude", reconnu par le filtre de pertinence), 1 rejet par déduplication d'id (collision interne, comportement attendu). Fiches de test retirées après vérification – rien n'est resté dans la base en attendant la validation du reste des 30 fichiers.

8.4 Fusion du brouillon dans la base de connaissances

Après revue du registre par l'utilisateur, fusion en masse du brouillon complet via `data/scripts/merge_programmes` – réutilise la **même** fonction de construction de fiche que `extraction_app` (`build_fiches_from_classes`) pour générer des id identiques : un futur ré-upload d'un de ces mêmes PDF via l'appli sera donc reconnu comme doublon plutôt que dupliqué. Dédup par id uniquement (pas de vérification sémantique TF-IDF ici – contenu entièrement nouveau, brouillon déjà relu par l'utilisateur, contrairement à une source externe non maîtrisée).

474 fiches ajoutées dans `site_esprit_clean.json` et `site_esprit.json` (13 collisions d'id internes au brouillon correctement déduplicquées, 0 collision avec la base existante). Testé après ré-indexation avec deux questions réelles :

- "Quelles sont les matières du panier Recherche Opérationnelle en 4DS ?" – réponse exacte et complète (3 matières, heures/période/ECTS corrects, total 5 ECTS, score 0,742), désambiguïsée correctement de la classe voisine "4DS Actuariat".
- "Combien d'ECTS vaut le module Architecture des SI I en 4 ERP-BI ?" – fiche correcte retrouvée en premier (score 0,722), mais réponse légèrement imprécise (cite le total ECTS du panier entier – 6 – plutôt que l'ECTS du module précis demandé – 2 – alors que l'information exacte est bien présente dans la fiche). Limite de génération, pas de récupération ni de donnée.

8.5 Constat : “liste toutes les matières” ne remontait qu’une partie

Constat : la question “Liste toutes les matières que j’étudie en 4ème ERP-BI” ne renvoyait que 11 matières sur 26 réelles. Cause : chaque panier (unité d’enseignement) est une fiche séparée – 13 fiches pour la seule classe “4 ERP-BI” – alors que `TOP_K = 5` limite structurellement le nombre de fiches transmises au LLM à chaque question. Pour une question “liste tout”, 8 des 13 paniers n’atteignaient jamais son contexte, quel que soit le prompt – pas un problème de génération.

Corrigé : ajout d’une fiche récapitulative dédiée par classe (`_build_liste_complete` dans `programme_etude_extractor.py`), groupant tous les paniers et matières d’une classe en un seul chunk dense, découpée en parties si elle dépasse 2000 caractères (aucune des 56 classes n’en a eu besoin après simplification du format). Générée par la même fonction partagée que les fiches par panier – s’applique donc aussi bien à un futur upload via `extraction_app` qu’à la fusion en masse.

50 fiches “liste complète” ajoutées (56 classes attendues, quelques doublons d’id internes dédupliqués). Question retestée après ré-indexation : les 13 paniers et 26 matières sont désormais listés, avec le total exact (26 matières, 60 ECTS) – contre 11 matières sur 5 fiches auparavant.

8.6 Rebond : confusion entre classes voisines (4e/5e année)

Constat : avec la formulation exacte de l’utilisateur (“Quels sont toutes les matières à étudier en 4ème ERP/BI ?”), la fiche “5 ERP-BI – Liste complète” remontait à la place de “4 ERP-BI – Liste complète” (score plus élevé), qui elle ne rentrait pas dans le top 5 – réponse à nouveau incomplète. Cause : les deux fiches ont un gabarit quasi identique, seul un chiffre (4 vs 5) les distingue – signal trop faible pour l’embedding, exactement la même classe de problème que la confusion d’année déjà rencontrée sur les tarifs de scolarité 2024-2025/2025-2026.

Corrigé : `_annee_disambiguation` ajoute désormais une phrase explicite (numéro + ordinal, ex. “Ceci concerne précisément la classe 4 ERP-BI, 4ème année – à ne pas confondre avec une autre année du même programme”) en tête de chaque fiche (panier et liste complète) dont le nom de classe contient un chiffre. Le script de fusion a été rendu idempotent par **upsert** (retire puis régénère toutes les fiches `upload_pdf_prog_*`) pour pouvoir appliquer une amélioration de génération sans laisser d’anciennes versions périmées en base.

524 fiches régénérées avec la désambiguation (même total, 1099 fiches). Question retestée après ré-indexation : les 5 sources sont désormais toutes “4 ERP-BI” (plus de confusion avec “5 ERP-BI”), la fiche liste complète en 2^e position (score 0,807), réponse correcte sur les 12 paniers et le total exact (60 ECTS).

8.7 Solution de fond : NLU léger et filtrage par métadonnée

Constat : la désambiguation par texte répété (section précédente) corrige bien le cas testé, mais reste fragile à la formulation exacte de la question – un rephrasage légèrement différent peut à nouveau faire remonter la mauvaise classe, puisque tout repose sur le poids qu’accorde l’embedding à un simple chiffre noyé dans un texte par ailleurs quasi identique. Le même problème structurel touche aussi le bug `TOP_K` (section “Constat : liste toutes les matières”) : une rustine de contenu (fiche récapitulative) aide, mais ne résout pas la cause racine – la recherche reste une similarité d’embedding non bornée par la structure réelle des données (une classe = un ensemble précis et connu de fiches).

Corrigé : ajout d'une couche de compréhension légère, à base de règles (pas de framework NLU/ML – cohérent avec la contrainte 100 % gratuit/local et avec un mécanisme déjà existant dans le projet, `modules/rag/programmes.py` et sa fonction `mentioned_programme`, qui ne retient un candidat que s'il est *seul* à correspondre, sans jamais deviner). Le nouveau module `modules/rag/nlu.py` ajoute deux détections :

1. `detect_classe_mention(question)` – normalise la question (accents, casse, séparateurs - / _, suffixes ordinaux 4ème/4eme) et la compare à la liste des classes connues, extraite dynamiquement des titres des fiches "Programme d'étude" déjà en base. Ne renvoie une classe que si **une seule** correspond dans la question – sinon `None`, et le comportement reste inchangé (même règle de prudence que `mentioned_programme`, jamais de clarification demandée en mode vocal).
2. `is_list_all_intent(question)` – détecte les formulations de type "liste toutes les matières", "tous les paniers", etc.

Ces deux signaux permettent de filtrer la recherche par **métadonnée** plutôt que de compter uniquement sur la similarité d'embedding : chaque fiche reçoit désormais un champ `classe` lors de l'indexation (`ingest.py`), et ChromaDB (déjà en place, `where=` supporté nativement par la version installée) filtre la recherche à ce sous-ensemble avant de calculer la similarité. Quand la question demande une liste exhaustive, `retrieve_all` récupère **toutes** les fiches de la classe via `collection.get(where=...)` – en s'affranchissant totalement de la limite `TOP_K`, plutôt que de dépendre d'une fiche récapitulative suffisamment bien classée. Le prompt envoyé au LLM reçoit dans ce cas une note explicite (`LIST_ALL_NOTE`) lui demandant d'énumérer tout le contexte fourni sans en résumer une partie. Portée : uniquement la catégorie "Programme d'étude" (524 fiches sur 1099, seule catégorie structurée en "une entité, plusieurs fiches" – confirmé, les autres catégories sont des FAQ plates) ; si aucune classe n'est détectée, le comportement est strictement identique à avant pour toutes les autres questions (calendrier, paiements, règlements...).

Quatre vérifications effectuées après ré-indexation (1099 fiches, nouveau champ `classe`) et redémarrage de l'API :

- Question "Quels sont toutes les matières à étudier en 4ème ERP/BI ?" – 14 sources renvoyées, **toutes** "4 ERP-BI" (zéro contamination "5 ERP-BI"), les 12 paniers et 26 matières correctement énumérés.
- Question ECTS précise ("Combien d'ECTS vaut le module Architecture des SI I en 4 ERP-BI ?") – 5 sources renvoyées, toutes "4 ERP-BI", réponse exacte.
- Non-régression : question calendrier déjà testée ("Quand commencent les études pour la 5ème année ?", aucune classe détectée) – mêmes fiches retournées qu'avant l'ajout du NLU, comportement inchangé.
- Cas ambigu : "4DS" seul se résout sans ambiguïté sur "Classe 4DS" (le nom "Classe 4DS Actuariat" n'est pas confondu, son motif complet n'apparaissant pas dans la question) ; "classe 4DS Actuariat" correspond en revanche aux deux classes à la fois – aucun filtre n'est appliqué (comme prévu), et la recherche globale retombe malgré tout sur les bonnes fiches grâce au mot "Actuariat" dans la question. Aucun plantage, aucune fausse demande de clarification dans les deux cas.

Les deux bugs sont donc corrigés à la racine, pour l'ensemble des classes du programme d'étude (pas seulement "ERP-BI"), et de façon indépendante de la formulation exacte de la question.

8.8 Signalement utilisateur : classes de 1ère à 3ème année sans réponse

Constat : peu après la mise en service du NLU, l'utilisateur signale que les questions sur les matières de 3ème année (classes "3A"/"3B") restent sans réponse correcte, et qu'il rencontre par ailleurs des erreurs de connexion. Deux causes distinctes, sans lien entre elles, ont été trouvées.

Cause 1 – étiquette de classe jamais extraite pour 6 fichiers. Contrairement aux ~50 autres PDF de programme d'étude (qui placent le nom de la classe explicitement dans le tableau, ex. "Classe 4DS Actuariat"), les fichiers `Plan_d'étude-1A/2A/2P/3A/3B.pdf` et `plan_d'étudeSLEAM2` utilisent une mise en page différente où cette information n'apparaît nulle part dans le tableau lui-même. `_find_classe_label` retombait alors sur un libellé sans rapport ("S3", "Semestre - 5", "page2_tableau1"...), rendant ces classes invisibles pour le NLU (filtrées comme "poubelle", voir `JUNK_CLASSE_PATTERN`) et leur contenu noyé dans la recherche globale sans étiquette fiable – exactement le symptôme rapporté (aucune classe détectée $\Rightarrow$ pas de filtre $\Rightarrow$ réponse tirée de la mauvaise année).

Corrigé : ajout d'un filet de secours (`_filename_classe_fallback` dans `extract_programmes_etude.py`) qui dérive le nom de classe du nom de fichier (ex. "Plan_d'étude-3A.pdf" $\rightarrow$ "3A"), appliqué **uniquement** si le fichier ne contient pas déjà plusieurs classes distinctes correctement détectées dans le tableau – condition qui exclut à raison `plan_d'étudeSLEAM2526.pdf` (2 classes réelles, "4 SLEAM" et "5 SLEAM", déjà bien détectées pour 16 de ses 30 fiches ; les 14 restantes gardent un libellé indéterminé, limite connue non résolue faute de moyen fiable de les rattacher à l'une ou l'autre sans deviner).

Cause 2 – cache local du modèle d'embedding incomplet sur le disque D. Sans rapport avec le NLU : le dossier `HF_HOME` redirigé vers le disque D (lors de la migration décrite en "Migration du disque C vers le disque D") contient un téléchargement **inachevé** du modèle d'embedding (fichier `model.safetensors` absent, blob `.incomplete` de 536 Mo) – le cache d'origine sur le disque C, lui, est complet. Selon le contexte (accès réseau autorisé ou non), ce cache cassé provoquait soit un blocage de plusieurs minutes (tentative de re-téléchargement sur un réseau lent), soit une erreur serveur immédiate – correspondant au "problème de connexion" signalé.

Corrigé : redémarrage du serveur **sans** la variable `HF_HOME` pointant vers le disque D (le cache par défaut sur le disque C, complet, est utilisé), avec `HF_HUB_OFFLINE=1` pour garantir qu'aucun appel réseau n'est tenté au chargement du modèle.

Base ré-extraite (1038 lignes, identique), ré-indexée (1089 fiches) et questions retestées en direct pour les 4 classes concernées :

- "1A" – 25 sources, toutes étiquetées "1A".
- "2A" – 26 sources, toutes étiquetées "2A".
- "3A" – 28 sources, toutes étiquetées "3A".
- "3B" – 17 sources, toutes étiquetées "3B", réponse listant correctement les 14 paniers.

Chaque classe répond désormais correctement, filtrée sur elle-même, sans contamination d'une autre année. Le serveur démarre et répond sans blocage réseau.

8.9 Signalement utilisateur : question ECTS sur “4MécaT” sans réponse

Constat : l'utilisateur signale qu'une question sur le panier ayant le plus d'ECTS en “4MécaT” obtient le message de repli (“je n'ai pas d'information précise”), et suppose que la casse/les accents de “MécaT” en sont la cause. Trois causes distinctes ont en réalité été trouvées, dont aucune n'est liée à la casse.

Vérification de la casse/accents : `nlu._normalize()` passe déjà le texte en minuscules puis retire les accents (décomposition NFKD + suppression des marques combinantes) avant toute comparaison – *avant* même ce signalement. Testé en direct : “4mecat” et “4MécaT” résolvent tous deux exactement à la même classe (`detect_classe_mention`), et les deux formulations produisent les 13 mêmes fiches en source. Ce n'était donc pas la cause – mais deux bugs bien réels expliquaient le symptôme.

Cause 1 – 3e colonne texte non gérée (perte de données + paniers fusionnés à tort). Le fichier PE-4MécaT.pdf (et dans une moindre mesure PE 1GC.pdf/PE 4GC.pdf) renvoie parfois le nom d'une matière trop longue dans la colonne voisine plutôt que dans la colonne “matière” habituelle (cellule vide, pas `None`, dans cette dernière sur ces lignes-là). `_classify_columns` ne prévoyait que 2 colonnes de texte (matière + panier) et prenait à tort cette 3e colonne pour le panier : le vrai panier (colonne 0, ex. “Automatisme Industriel”, “Systèmes embarqués”...) était ignoré, et les lignes dont la matière n'apparaissait QUE dans cette colonne de repli étaient silencieusement rejetées (matière vide $\Rightarrow$ ligne ignorée). Résultat pour “4MécaT” : 13 paniers réels réduits à 2 fiches fourre-tout, 6 matières sur 30 perdues.

Corrigé : une colonne de texte supplémentaire n'est traitée comme panier que si son remplissage ne coïncide PAS systématiquement avec une matière vide sur les mêmes lignes – sinon elle est reconnue comme un simple repli de la colonne matière et fusionnée dedans (`matiere_overflow_col`). Ré-extraction : 1038 $\rightarrow$ 1049 lignes matière au total (+11, dont +6 pour “4MécaT” seul), panier correctement attribué aux 13 groupes réels de “4MécaT”.

Cause 2 – questions de comparaison non reconnues comme “liste tout”. “Quel panier a le plus d'ECTS” ne contient aucun des mots-clés de `is_list_all_intent` : la recherche restait donc limitée à TOP_K=5 fiches par similarité, et aucune fiche individuelle ne “ressemble” sémantiquement à une question de comparaison (aucun texte de fiche ne dit “je suis le panier avec le plus d'ECTS”) – score maximal observé 0,435, sous le seuil de réponse.

Corrigé : ajout de motifs superlatifs (“le plus élevé”, “le plus haut”, “le maximum”...) à `is_list_all_intent`, qui déclenchent désormais le même `retrieve_all` que pour une liste exhaustive – logique, une comparaison exige aussi de voir tous les paniers.

Cause 3 – seuil de score inadapté au chemin “liste tout”. Même après correction de la cause 2, le score de la meilleure fiche parmi les 13 restait sous le seuil `SCORE_THRESHOLD=0,45` (aucune fiche isolée ne “ressemble” à une question de comparaison), déclenchant quand même le repli. Or la classe a déjà été identifiée de façon sûre et symbolique (nom de classe reconnu, pas une similarité floue) – le seuil de score, pensé pour détecter un sujet hors-base, n'a pas de sens dans ce cas.

Corrigé : le seuil de score est désormais ignoré quand `list_all` est actif (`pipeline.py`).

Les trois corrections combinées permettent enfin une réponse correcte, testée avec les deux formulations (“4mecat” et “4MécaT”) : les 13 fiches de la classe sont récupérées dans les deux cas, avec les mêmes sources et scores. Limite résiduelle observée et assumée : le LLM local (qwen2.5:7b) fait parfois une erreur de calcul en comparant les ECTS de 13 paniers dans le contexte (une réponse a cité “Projet CDIO” à 8 ECTS comme le plus élevé, une autre a conclu à tort “Systèmes embarqués” à 6 ECTS après avoir listé “Projet CDIO” à 8 – incohérence de raisonnement du modèle, pas un problème de récupération : les fiches sources sont identiques et correctes dans les deux cas). Amélioration possible mais non retenue pour l’instant : demander explicitement au modèle de comparer chiffre par chiffre avant de conclure.

8.10 Garde-fous obligatoires avant sauvegarde

Deux vérifications s’appliquent à toute fiche candidate, quelle que soit sa source, avant toute fusion dans la base :

1. **Filtrage de pertinence** (`services/relevance_filter.py`) – rejette une fiche si ni son titre ni son contenu ne contiennent de mot-clé d’un des domaines couverts par l’assistant (admissions, scolarité, règlements, examens, calendrier, paiements, vie étudiante, stages, programmes, contacts).
2. **Vérification sémantique** (`services/semantic_dedup.py`) – comparée contre le contenu de **toutes** les fiches déjà présentes dans la base (pas seulement même catégorie/source). Seuil configurable en haut de `config.py` (`SIMILARITY_THRESHOLD = 0.92`).

Une dedup complémentaire par id empêche par ailleurs tout écrasement d’une fiche existante d’une autre source.

Migration TF-IDF → embeddings (sur demande explicite : “je veux que le score soit par le sens”) : la vérification sémantique utilisait initialement TF-IDF + similarité cosinus (scikit-learn), une méthode lexicale (compte de mots partagés) qui sous-notait deux fiches disant la même chose avec des mots différents. Remplacée par de vrais embeddings – `modules/rag/embeddings.py`, modèle paraphrase-multilingual-mpnet-base-v2, le **même** que celui utilisé par la recherche du RAG lui-même (même espace vectoriel). Le seuil a dû être recalibré à cette occasion (0,85 → 0,92) : la distribution des scores diffère entre les deux méthodes – avec des embeddings, deux fiches sans rapport peuvent déjà dépasser 0,8 de similarité, un seuil resté à 0,85 aurait donc généré des faux positifs.

8.11 Interface de validation (/a-valider)

Toute extraction **non supervisée** (les deux jobs planifiés, voir section suivante) n’est jamais fusionnée directement dans la base : elle atterrit dans une file d’attente que l’admin doit valider fiche par fiche (ou lot par lot) sur `/a-valider` – contrairement à un upload manuel (PDF/Excel/Word/image/URL via l’interface), où le geste de l’admin constitue déjà lui-même l’approbation explicite.

Aperçu de similarité, sans écriture : pour chaque fiche candidate de chaque lot en attente, la page calcule – **avant** toute décision de l’admin – si elle serait jugée doublon si elle était approuvée maintenant, en comparant son contenu à l’**intégralité** de la base actuelle (mêmes embeddings et même seuil que les garde-fous, section 8.10). Le calcul se fait en **un seul appel d’embedding groupé** pour tous les candidats de la page (pas un appel par fiche), et le comparateur construit sur toute la base est mis en cache (invalidé sur changement de la base), pour éviter de ré-embedder ~800 fiches à chaque chargement de page (~65s sans

ce cache). Chaque candidat affiche soit “nouveau contenu” (avec son meilleur score), soit la fiche existante la plus proche – titre **et** contenu complet affichés côte à côte, pour que l’admin compare visuellement plutôt que de se fier au score seul – soit “serait rejeté (hors-sujet)” si le filtre de pertinence échoue déjà.

Masquage des doublons exacts (score = 1) : sur demande explicite (“je veux que je puisse supprimer les questions... la colonne fiche similaire n’est pas utile” initialement, puis affinée), les candidats dont le score de similarité vaut exactement 1 (doublon confirmé) sont masqués de l’affichage – inutile de les montrer, ils ne font qu’allonger une liste mensuelle déjà longue. **Masquage d’affichage uniquement** : la fiche reste présente dans les données du lot et reste rejetable/approuvable via les actions groupées ci-dessous.

Recherche dynamique : une barre de recherche filtre les fiches affichées en temps réel (à la frappe, sans rechargement de page) – JavaScript pur, comparaison de sous-chaîne sur le titre et le contenu déjà rendus dans la page (pas de requête serveur, pas de recherche sémantique dans la barre elle-même – différent du score de similarité ci-dessus).

Actions : approbation/rejet fiche par fiche, ou en bloc par lot (“Tout approuver” / “Tout rejeter” – les deux garde-fous de la section 8.10 restent appliqués fiche par fiche même en approbation groupée). Une fiche approuvée est écrite directement dans `site_esprit_clean.json` **et** `site_esprit.json`, et journalisée dans l’historique (section suivante).

8.12 Réindexation à la demande

En approuvant une fiche, elle devient présente dans `site_esprit_clean.json` mais **pas encore cherchable** par l’assistant : la base vectorielle ChromaDB n’est jamais reconstruite automatiquement après chaque clic (effet de bord jugé inacceptable – une ré-indexation complète prend de l’ordre d’une minute). Un bandeau en haut de `/a-valider` indique si l’index est à jour ou obsolète, en comparant la date de dernière modification de `site_esprit_clean.json` à celle du fichier ChromaDB – et un bouton “Réindexer maintenant” déclenche `python -m modules.rag.ingest` en sous-processus séparé (pas un import direct dans le processus `extraction_app`, pour éviter de charger `sentence-transformers/torch` en permanence dans une application qui n’en a besoin que rarement). Le résultat du dernier lancement (succès/échec, sortie console) reste affiché dans le bandeau jusqu’au prochain redémarrage du serveur.

8.13 Page d’historique (`/history`)

Page en lecture seule (pas de recherche ni filtre, contrairement à `/a-valider`) listant chronologiquement (plus récent en premier) chaque extraction – manuelle ou planifiée : date, source, type, origine (manuel/planifié), nombre de fiches ajoutées, rejetées pour hors-sujet, rejetées pour doublon (avec le détail dépliable de chaque paire *titre candidat* ~ *titre existant (score)*), et statut (erreur le cas échéant, “à vérifier manuellement”, ou ok).

8.14 Authentification et planificateur

Un compte unique protège l’interface (mot de passe généré aléatoirement au premier lancement, affiché une seule fois en console, jamais versionné). Deux jobs APScheduler, tous deux **mensuels** (sur demande explicite – initialement quotidien + mensuel, simplifié en mensuel uniquement pour les deux) :

Job	Fréquence	Action
monthly_url_rescraper	1 ^{er} du mois, 03h00	Rejoue l'extraction pour chaque source de type URL déjà soumise via l'interface.
monthly_options_scraper	1 ^{er} du mois, 04h00	Re-scrape les options de spécialisation ESPRIT Tunis.

Les deux heures sont décalées d'une heure entre elles (heures creuses). Ni l'un ni l'autre ne rejoue les sources PDF/Excel/image uploadées manuellement, dont le fichier d'origine n'est pas conservé. Le BackgroundScheduler tournant en mémoire du processus `extraction_app`, un job ne se déclenche que si le serveur est effectivement en cours d'exécution à cette date/heure précise.

Aucun des deux jobs n'écrit jamais directement dans la base – les deux résultats sont systématiquement mis en attente sur `/a-valider` (section 8.11), jamais fusionnés automatiquement, contrairement à un upload manuel où le geste de l'admin vaut déjà approbation.

8.15 Intégration avec le dashboard React

Le dashboard React (`dashboard/`) et `extraction_app` restent deux processus **entièrement séparés** (ports 8000 vs 8001, authentification indépendante). La page "Administration" du dashboard (`dashboard/src/pages/Admin.tsx`) n'affiche **aucune** donnée d'`extraction_app` elle-même – c'est un simple point d'entrée avec des liens directs vers `/`, `/a-valider` et `/history` sur le port 8001, sans appel API ni partage de session entre les deux applications.

8.16 Découverte : corruption d'encodage dans les fichiers .docx d'admission

En testant le nouveau pipeline Word sur `FAQ admission cours du soir.docx`, une section absente de la base (« Modalités Financières et Annulations ») a été correctement identifiée comme nouvelle par la vérification sémantique – mais son contenu s'est révélé truffé du caractère de remplacement Unicode `U+FFFD` () à la place de tous les accents français (« *Régler mes frais de scolarité* »). Vérification faite en lisant le `.docx` source directement avec `python-docx` (sans passer par `extraction_app`) : la corruption est bien dans le fichier source lui-même, pas introduite par le nouveau pipeline. Les 4 sections de ce même document déjà fusionnées en base via `data/scripts/extract_admission_docx.py` lors d'un travail antérieur présentent exactement la même corruption – ce n'est donc pas une régression, mais un défaut préexistant qui touche déjà des fiches en production.

`U+FFFD` signale une perte de données irréversible (un octet n'a pas pu être décodé à un moment donné et a été remplacé) – aucune astuce de réencodage ne peut récupérer les caractères d'origine. La correction nécessite d'obtenir une copie propre (non corrompue) des fichiers `.docx` concernés, pas un correctif côté code. Fiches concernées à ré-extraire une fois une copie saine disponible : les 4 sections déjà en base (`admission_docx_faq_admission_cours_du_soir_01/02/03/05`) et la section manquante détectée (04), et potentiellement d'autres fichiers du dossier `data/raw_data/Admission/` non encore vérifiés un par un.

8.17 Limites connues de l'extraction des programmes d'étude

- Plan d'étude IoSyS 2526.pdf ne contient **aucun texte extractible** : pdfplumber détecte bien la grille du tableau (lignes vectorielles) mais chaque cellule est vide, sur les 3 pages – signe d'un export où le texte a été converti en tracés (outlines) ou d'une image aplatie, pas d'un vrai texte encodé. Aucune méthode de lecture de texte ne peut y remédier ; il faudrait soit une copie du fichier avec un texte natif, soit de l'OCR (non implémenté, fiabilité incertaine sur un tableau dense). Décision : laissé de côté pour l'instant – le modèle répond honnêtement qu'il n'a pas l'information plutôt que d'inventer.
- Les classes de Génie Civil (PE 1/2/3/4/5GC.pdf) n'ont pas de colonne ECTS du tout dans leurs tableaux sources – champ laissé vide plutôt que fiche rejetée (voir `_find_header_row_indices`).
- plan_d'étudeSLEAM2526.pdf regroupe 2 classes réelles ("4 SLEAM"/"5 SLEAM") dans un seul fichier ; ~13 lignes matière dont la classe n'a pas pu être détectée dans le tableau restent sans étiquette fiable (non rattachées par filet de secours nom-de-fichier, car ce fichier contient plusieurs classes – voir section précédente).

8.18 Limites assumées

- Le filtrage de pertinence est une simple correspondance de mots-clés, pas une compréhension du sens – peut laisser passer du hors-sujet ou rejeter du contenu pertinent mal formulé.
- La vérification sémantique (TF-IDF/cosinus) est lexicale, pas une vraie compréhension sémantique comme les embeddings du RAG – deux fiches formulées très différemment peuvent passer sous le seuil et être dupliquées.
- Le scraping générique ne peut ni exécuter de JavaScript ni contourner un blocage anti-bot ; si le contenu récupéré est vide ou anormalement court, aucune fiche trompeuse n'est sauvegardée – un avertissement explicite est affiché à la place.
- La dédup sémantique recalcule le vectoriseur TF-IDF sur toute la base à chaque extraction – viable à l'échelle actuelle (~600 fiches), mais à surveiller si la base grossit à plusieurs milliers de fiches (piste d'optimisation non implémentée : vecteurs pré-calculés).

Validation effectuée : les cinq pipelines testés de bout en bout via l'interface – Excel (fiche pertinente acceptée, fiche hors-sujet rejetée), PDF (ré-extraction d'un règlement déjà en base : 23 sections correctement détectées comme doublons, zéro ajout), Word (ré-extraction d'un FAQ admission déjà partiellement en base : 4 sections détectées comme doublons, 1 section manquante correctement identifiée comme nouvelle – voir découverte de corruption d'encodage ci-dessus), URL (page `esprit.tn` et site générique hors-sujet), et image (échec propre et message clair en l'absence du binaire Tesseract, sans plantage). Dédup par `id` vérifiée sur un ré-upload exact du même fichier. Job planifié déclenché manuellement sans erreur. `site_esprit_clean.json` confirmé non modifié par l'application.

9. Dashboard qualité et supervision (/qualite)

9.1 Contexte et objectif

L'encadrant a fourni 3 fichiers de 75 questions chacun (français, arabe standard, arabe tunisien – 225 questions au total), **sans réponses de référence**. Objectif : mesurer objectivement la qualité du modèle malgré cette absence, et donner à l'admin un tableau de bord pour suivre l'évolution dans le temps.

Décision de conception clé : en l'absence de réponse de référence, le “% correct” ne peut pas être calculé automatiquement. Le dashboard combine donc deux approches complémentaires plutôt que d'en choisir une seule : des **indicateurs objectifs automatiques** (taux d'échec, score de confiance, cohérence multilingue – voir plus bas) calculés sur **tous** les résultats, et une **annotation manuelle** correct/incorrect que l'admin remplit lui-même – le pourcentage de bonnes réponses n'est alors calculé QUE sur les réponses annotées, jamais extrapolé sur le reste.

9.2 Indicateurs calculés

- **Taux d'échec (fallback)** : proportion de questions où aucune fiche pertinente n'a été trouvée (score sous le seuil).
- **Score de confiance moyen** : score de similarité de la meilleure fiche récupérée, moyenné.
- **Cohérence multilingue** : similarité sémantique (embeddings) entre la réponse arabe standard/tunisienne et la réponse française pour la **même** question – proxy d'exactitude en l'absence de référence humaine (la base étant majoritairement en français, une réponse arabe très différente de la française est suspecte). Calculable uniquement pour les questions du test officiel (posées dans les 3 langues à la fois), pas pour le trafic réel.
- **% correct annoté** : calculé uniquement sur les réponses que l'admin a explicitement marquées correct/incorrect.

9.3 Visualisations variées (compétence *dataviz*)

Une première version du dashboard utilisait le même graphique en barres pour les 5 indicateurs – retour utilisateur explicite : trop uniforme, pas assez “tableau de bord professionnel”. Le dashboard a été refondu pour utiliser un type de visualisation différent selon le *job* de chaque donnée (méthode issue de la compétence interne *dataviz*, pas une variété arbitraire) :

- **Tuiles KPI** (chiffres-clés) pour les indicateurs globaux les plus importants (tests exécutés, taux d'échec global, score de confiance moyen, % correct), avec écart signé vs un run “avant corrections” sauvegardé.

- **Jauges (*meters*)** pour les 4 indicateurs qui sont des ratios contre une limite (taux d'échec, score de confiance, % correct, cohérence) – la couleur y encode une **sévérité** (bon/moyen/à surveiller), pas l'identité de la langue, contrairement aux autres graphiques.
- **Graphique avant/après (*dumbbell*)** : comparaison visuelle entre un run de résultats sauvegardé avant une série de corrections et le run courant, par langue – seul le temps de réponse reste en simple barres (magnitude sans limite naturelle, pas un ratio).

Palette catégorique (identité de langue) validée par le script de vérification de la compétence *dataviz* contre le fond réel de l'application : séparation entre paires adjacentes suffisante pour un daltonisme protanope/deutéranope.

9.4 Journalisation automatique du trafic réel

Sur demande explicite, le dashboard ne se limite plus au test des 75 questions fournies : **chaque question réellement posée** via l'application (`/api/ask`, `/api/voice-ask`, `/api/converse`) est désormais journalisée automatiquement, avec sa langue détectée, mélangée dans la même liste et les mêmes indicateurs que le test officiel.

Détection de langue du trafic réel (`modules/rag/translate.py:: detect_text_language`) : français/arabe fiable (alphabet différent) ; la distinction arabe standard/tunisien est une **heuristique par mots-clés** (liste de marqueurs très caractéristiques du tunisien, ex. *شانون*, *قداش*, *باهي*) assumée comme approximative – aucune vraie détection de dialecte n'existe de façon fiable pour du texte court.

Architecture : les fonctions de lecture/écriture du journal ont été extraites dans un nouveau module neutre `modules/quality_log.py`, partagé entre `backend` et `extraction_app`, plutôt que de faire dépendre le backend de l'application d'administration. L'écriture se fait en tâche de fond (`BackgroundTasks` de `FastAPI`) pour ne jamais ralentir la réponse renvoyée à l'appelant.

Bug de concurrence corrigé : plusieurs vraies questions peuvent arriver en même temps (contrairement au script de test, séquentiel) – un verrou (`threading.Lock`) protège désormais toute écriture du fichier de résultats, qu'il s'agisse d'un ajout (trafic réel) ou d'une réécriture complète (annotation), pour éviter qu'une écriture concurrente n'en écrase silencieusement une autre. Une corruption réelle (octets NUL dans une ligne) avait déjà été observée avant ce correctif, causée par une collision entre deux processus lors d'un redémarrage rapide.

Sur demande explicite, les tours où le modèle demande une précision (§ 5.22, `needs_clarification=true`) ne sont **pas** journalisés – ce n'est pas une réponse à évaluer, seul le tour final (question re-combinée + vraie réponse) est conservé.

9.5 Annotation et suppression en temps réel (AJAX)

Les boutons Correct/Incorrect envoyaient initialement un formulaire classique, rechargeant toute la page `/qualite` et ramenant le défilement en haut – pénible pour annoter 225+ réponses à la suite. Remplacé par un appel `fetch()` : le serveur renvoie le nouvel état **et** tous les indicateurs recalculés en JSON, que le JavaScript client applique directement aux tuiles/tableau/jauges concernés, sans jamais recharger la page.

Un bouton **Supprimer** par question (avec confirmation) a été ajouté sur la même demande, permettant à l'admin de retirer une entrée non pertinente du journal – suit exactement le même mécanisme de mise à jour en temps réel que l'annotation (fonction partagée `_dashboard_payload()` côté serveur, réutilisée par les deux routes).

Le graphique avant/après (comparaison à un run figé) n'est volontairement jamais recalculé par ces mises à jour en temps réel – il compare à un snapshot immuable, une annotation ou une suppression ne le fait jamais changer.

10. Tests et validation

10.1 Tests manuels effectués

- `/api/health`, `/api/ask`, `/api/transcribe`, `/api/voice-ask`, `/api/speak`, `/api/converse` : tous testés de bout en bout avec succès.
- Enregistrements vocaux réels (français et arabe tunisien) via `tests/record_voice.py` puis upload manuel dans `/docs`.
- Cas limites du RAG testés : questions précises, questions vagues, questions sur des services généraux – comportement validé dans les trois cas après corrections.

10.2 Test systématique sur 225 questions (`scripts/run_quality_test.py`)

Rejeu automatique des 75 questions fournies par l'encadrant dans les 3 langues (225 tests) à travers le pipeline RAG réel, résultats persistés au fur et à mesure (voir chapitre 9). Méthodologie de comparaison avant/après une série de corrections : le fichier de résultats est sauvegardé avant de relancer le test complet, permettant de mesurer objectivement l'effet des correctifs plutôt que de se fier à l'impression subjective. Sur la dernière série de corrections (glossaire de traduction, bug NLU de classe fantôme, fiches KB manquantes) : taux d'échec global passé de 4,0% à 1,8%, cohérence tunisien↔français passée de 0,722 à 0,780.

10.3 Scripts de test disponibles

- `tests/test_rag_pipeline.py` – test interactif du RAG seul.
- `tests/test_stt_pipeline.py` – test interactif micro → transcription.
- `tests/record_voice.py` – enregistrement seul (sans transcription), pour tester ensuite via l'API.
- `tests/test_language_quality.py` – protocole structuré de test qualité arabe tunisien vs français (8 scénarios, rapport JSON sauvegardé).

11. Limites connues et travail restant

11.1 Limites techniques actuelles

- **Vitesse** : ~20 à 40 secondes par réponse vocale complète, due au GPU limité (4 Go VRAM) partagé entre Whisper (CPU) et Ollama (50% CPU / 50% GPU).
- **Qualité arabe tunisien** : limite structurelle des modèles gratuits disponibles (Whisper et Piper), documentée par la recherche académique – partiellement contournée côté génération en forçant systématiquement la réponse en arabe standard plutôt qu'en dialecte.
- **Espace disque C** : cause identifiée (Docker + pagefile Windows) mais correctif pas encore appliqué (nécessite droits admin + redémarrage).
- **Détection dialectale du trafic réel** : la distinction arabe standard vs tunisien pour la journalisation qualité (§ 9.4) est une heuristique par mots-clés, pas une vraie détection de dialecte – assumé comme approximatif.
- **Collision d'embeddings sur certaines formulations** : une formulation informelle précise peut rester mal retrouvée par la recherche sémantique malgré un contenu correct en base (ex. “frais d'inscriptions... (cours du jour)”, voir chapitre 5) – un filtrage par métadonnée “programme” résoudrait ce type de cas mais n'est pas encore implémenté.

11.2 Travail restant

- **Docker** : conteneurisation des services + déploiement – dossier vide.
- **Tests arabe approfondis** : lancer `test_language_quality.py` avec de vrais enregistrements et analyser les résultats.
- **Durcissement API REST** : gestion d'erreurs, tests automatisés.
- **Filtrage de recherche par programme** : étendre le filtrage par métadonnée (déjà en place pour “classe” et “campus”) à l'axe “programme” (cours du jour/soir, EMBA...), pour fiabiliser la désambiguïsation au-delà du réglage actuel de sensibilité.
- **/api/converse** : canal audio-vers-audio pas encore branché à un véritable système téléphonique ; n'a pas reçu la clarification interactive (§ 5.22), faute de mécanisme de mémoire de conversation adapté à ce canal.

Qualité des données (clôturé) : audit complet de `site_esprit_clean.json` terminé – fiches “gouvernance” et contacts restants corrigés, catalogue RDI hors-sujet supprimé, préfixe d’interface parasite retiré, doublons de contenu fusionnés, conflit des calendriers multi-promotions résolu, grilles tarifaires extraites et intégrées, titres “nom de fichier + page” nettoyés, champ `type` mort supprimé, détection d’ambiguïté rendue robuste au bruit de score, fiches de frais de scolarité fragmentées fusionnées. Base finale : 792 fiches.

Dashboard qualité (clôturé) : tableau de bord `/qualite` construit de zéro – indicateurs objectifs automatiques, annotation manuelle correct/incorrect en temps réel (AJAX, suppression incluse), visualisations variées (tuiles KPI, jauges, graphique avant/après), journalisation automatique du trafic réel mélangée au test officiel. Voir chapitre 9.

Clarification interactive (clôturé) : le mécanisme de détection d’ambiguïté entre programmes, déjà implémenté mais jamais activé, est désormais actif sur le chat et le vocal de démonstration, avec mémoire côté client pour recombinaison la question d’origine et la précision de l’utilisateur. Voir § 5.22.